

An abstract digital landscape illustration on the left side of the slide. It features a dark teal background with light green and yellow wavy lines representing a terrain or data flow. Overlaid on this are numerous small circles in various colors (green, yellow, orange, white) and thin white lines that resemble circuitry or neural network connections. Some circles are solid, while others are outlines. The overall style is modern and tech-oriented.

Dynamic Programming

Efficient Problem Solving Through Optimal Substructure

Anuska Nath | 002311001003

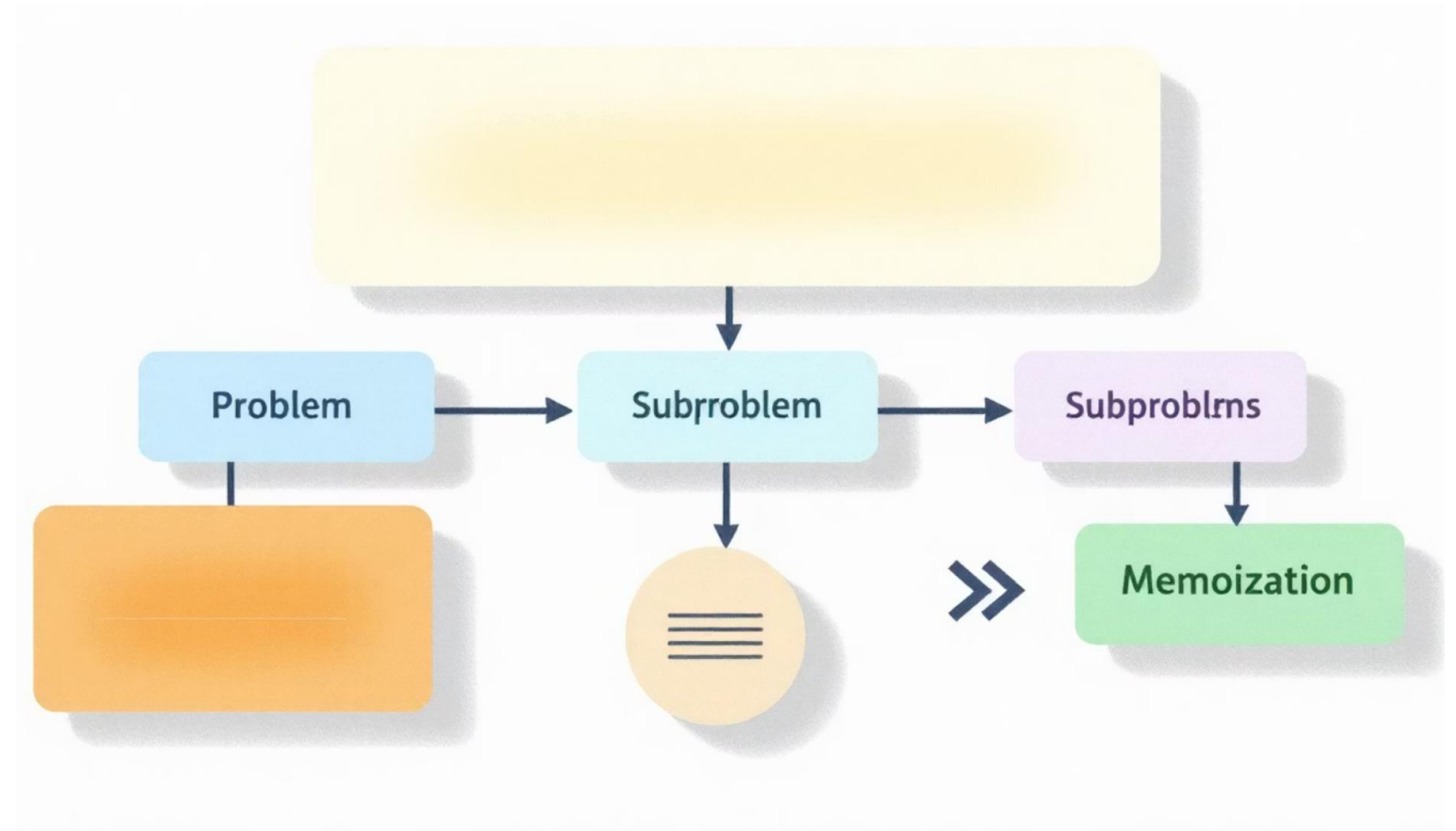
Information Technology - UG3

What is Dynamic Programming?

Dynamic Programming solves problems by storing solutions of smaller subproblems and reusing them.

Dynamic Programming (DP) is a powerful algorithmic technique that solves complex problems by breaking them into simpler overlapping subproblems. It stores solutions to subproblems to avoid redundant computations, dramatically improving efficiency.

Key characteristics: **optimal substructure** (optimal solution contains optimal solutions to subproblems) and **overlapping subproblems** (same subproblems solved multiple times).



Two Approaches to DP



Top-Down (Memoization)

Solve problem recursively, storing results of subproblems in a cache. Check cache before computing to avoid redundant work.

- Intuitive and natural
- Lazy evaluation
- Uses recursion



Bottom-Up (Tabulation)

Build solutions iteratively from smallest subproblems to the final solution. Fill table systematically.

- Space efficient
- No recursion overhead
- Iterative approach

Fibonacci Series: The Classic Example

The Fibonacci sequence demonstrates DP beautifully—each number is the sum of the two preceding ones: $F(n) = F(n-1) + F(n-2)$.

01

Naive Recursive Approach

$O(2^n)$ time complexity—exponential growth with massive redundancy in calculations.

02

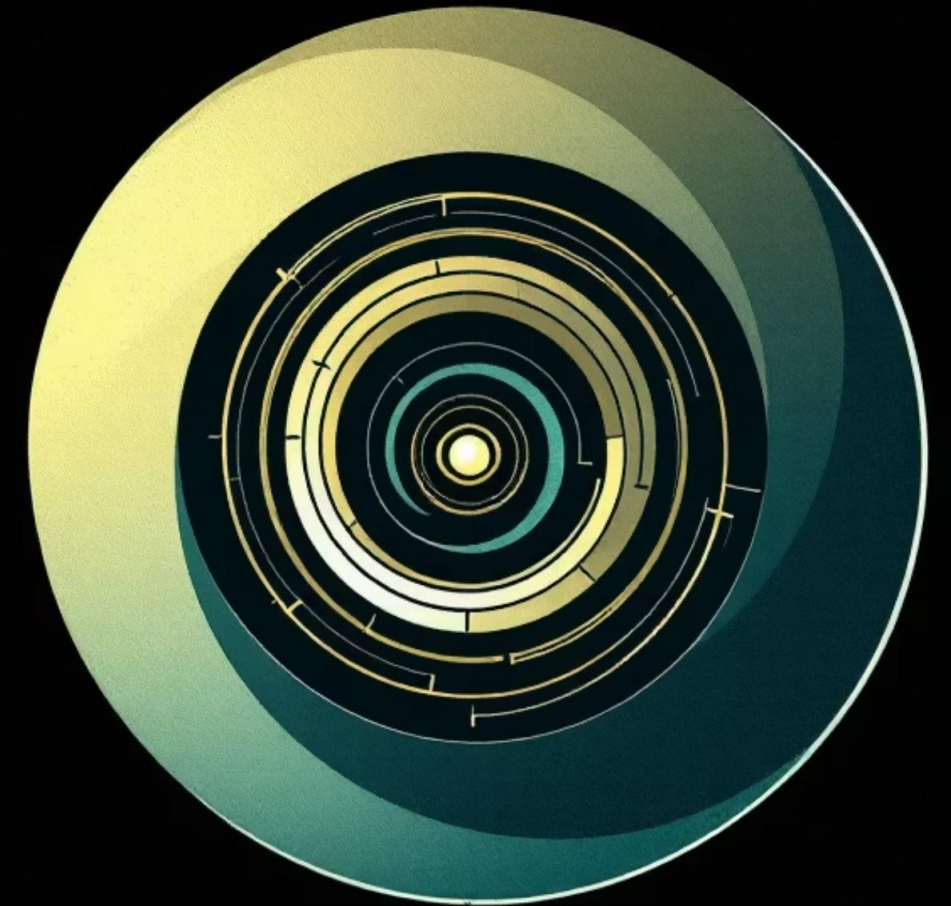
DP with Memoization

$O(n)$ time, $O(n)$ space—store computed values, eliminate redundant recursive calls.

03

Optimized Tabulation

$O(n)$ time, $O(1)$ space—iterative solution using only two variables to track previous values.



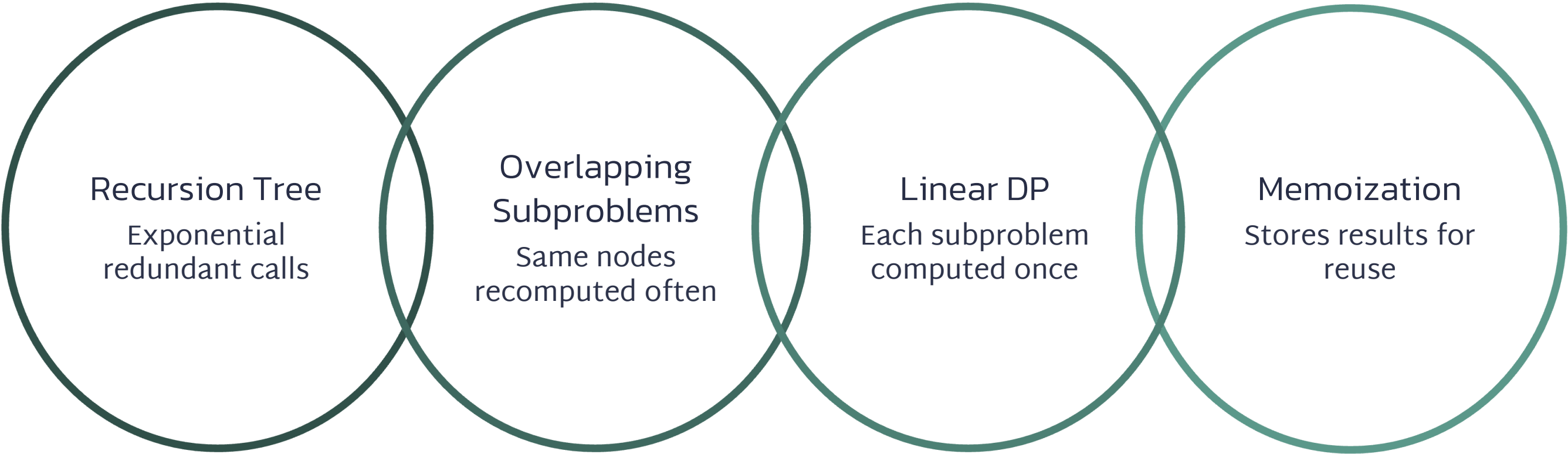
Visualizing Fibonacci with DP

Without DP

Tree diagram shows exponential growth of redundant recursive calls.
Same subproblems calculated repeatedly.

With DP

Linear path shows efficient computation. Each subproblem solved exactly once and stored for future reference.



The dramatic difference in computational efficiency is immediately apparent when visualizing both approaches side by side.



The Knapsack Problem

Given a knapsack with capacity W and n items, each with weight w_i and value v_i , maximize total value without exceeding weight capacity.

Decision at Each Item

Include or exclude current item based on which choice yields higher value

State Definition

$DP[i][w]$ = maximum value using first i items with weight limit w

Knapsack: Building the Solution



Initialize Base Cases

$DP[0][w] = 0$ for all w (no items means zero value)



Iterate Through Items

For each item, consider both inclusion and exclusion scenarios



Apply Recurrence Relation

$DP[i][w] = \max(DP[i-1][w], DP[i-1][w-w_i] + v_i)$

- $DP[i-1][w] \rightarrow$ item not included
- $DP[i-1][w-w_i] + v_i \rightarrow$ item included



Extract Final Answer

$DP[n][W]$ contains maximum achievable value



UNDERSTANDING 0/1 KNAPSACK PROBLEM & DYNAMIC PROGRAMMING SOLUTION

1. THE EXAMPLE PROBLEM (INPUT)



Max Capacity (C) = 6 kg

Item		W_i	V_i
Item 1		1kg	1
Item 2		2kg	4
Item 3		3kg	6
Item 4		4kg	8

Goal: Maximize total value within capacity.
0/1: An item cannot be split.

Abbreviations:

W = Weight
V = Value

Max Capacity (C) = 6 kg

Goal: Maximize total value within capacity.
0/1: Item cannot be split.
0/1: An item cannot be split.

2. THE DYNAMIC PROGRAMMING APPROACH

$$DP[i][j] = \max(DP[i-1][j], V_i + DP[i-1][j - W_i])$$

i = current item considered

j = remaining capacity



Subproblems: Build solutions to smaller problems in a table.



Decision for cell $[i][j]$:

- Take item / OR
- Do NOT take item i ?



Case 1: if $W_i > j$:

$$DP[i][j] = DP[i-1][j]$$



Case 2: if $W_i \leq j$:

Choose $\max(\text{Don't take, Take})$

3. FILLING THE DP TABLE (STEP-BY-STEP)

		Capacity j								
Items i	i	0	0	1	2	3	4	5	6	
	None									
	$i=0$ (no items)	0	0	0	0	0	0	0	0	
	Item 1 (1,1)	0	1	1	1	1	1	1	1	
	Item 2 (2,4)	0	1	1	4	5	5	5	5	
	Item 3 (3,6)	0	1	1	4	6	7	10	11	
Item 4 (4,8)		0	1	1	4	6	8	10	12	
		Capacity j								

For this cell for example:

$$DP[3][5] = \max(DP[2][5], V_3 + DP[2][5-3=2]) = 10$$

4. FINAL SOLUTION & BACKTRACKING

MAXIMUM VALUE: 12

SELECTED ITEMS:

4 → 2 → 4



Item 4 (4kg, 8)



Item 2 (2kg, 4)

Verification:

Total Weight = 6kg
Total Value = 12

Complexity Analysis

$O(nW)$

Time Complexity

n items $\times$ W capacity states

$O(nW)$

Space Complexity

2D table storing all states

$O(W)$

Optimized Space

1D array using only previous row

 **Key Insight:** Pseudo-polynomial time—efficient when W is reasonable but becomes impractical for large capacities.

Real-World Applications



Resource Allocation

Optimize budget distribution across projects, investment portfolio management, and capital budgeting decisions.



Bioinformatics

Sequence alignment, protein folding prediction, and genetic sequence analysis algorithms.



Operations Research

Solve cutting stock problems, cargo loading optimization, and production scheduling in manufacturing.



Game Development

AI decision trees, pathfinding optimization, and resource management in strategy games.

Key Takeaways



Store, Don't Recompute

Memoization eliminates redundant calculations by caching subproblem solutions



Build Bottom-Up

Iterative tabulation often more efficient than recursive memoization



Recognize Patterns

Optimal substructure and overlapping subproblems signal DP applicability

